

Document Search AI Agent (RAG-Powered)

An AI-powered Agentic RAG chatbot that enables users to upload PDFs, DOCX, and text documents, intelligently retrieve and reason over their contents, and answer complex natural language queries with accurate responses. The system supports automated document ingestion, semantic search, and automatic re-indexing of updated documents through file-change monitoring.

Tech Stack – Document Search AI Agent (Agentic RAG)

Backend

- Python
- FastAPI

Agent Framework

- LangGraph (for agentic workflows, query rewriting, retrieval validation, and decision-making)
- LangChain (for document loaders, retrievers, and integrations)

LLM

- Google Gemini 2.5 Pro

Embeddings

- BGE-small-en-v1.5

Vector Database

- ChromaDB (MVP/Development)
- Pinecone

Document Processing

- PyMuPDF (PDF)
- python-docx (DOCX)
- python-pptx (PPT/PPTX)
- openpyxl (Excel)
- BeautifulSoup (HTML)
- markdown (Markdown files)
- extract-msg (.msg files)

Storage & Version Control

- GitHub Repository for storing all source documents
- PyGithub and GitHub API for document synchronization and version tracking

Re-indexing & Monitoring

- Watchdog for detecting file changes
- Automated re-indexing pipeline for updated documents

Database

- PostgreSQL (chat history, metadata, audit logs, user sessions)

Frontend

- React.js (recommended)
- Streamlit (optional for rapid prototyping)

Core Features

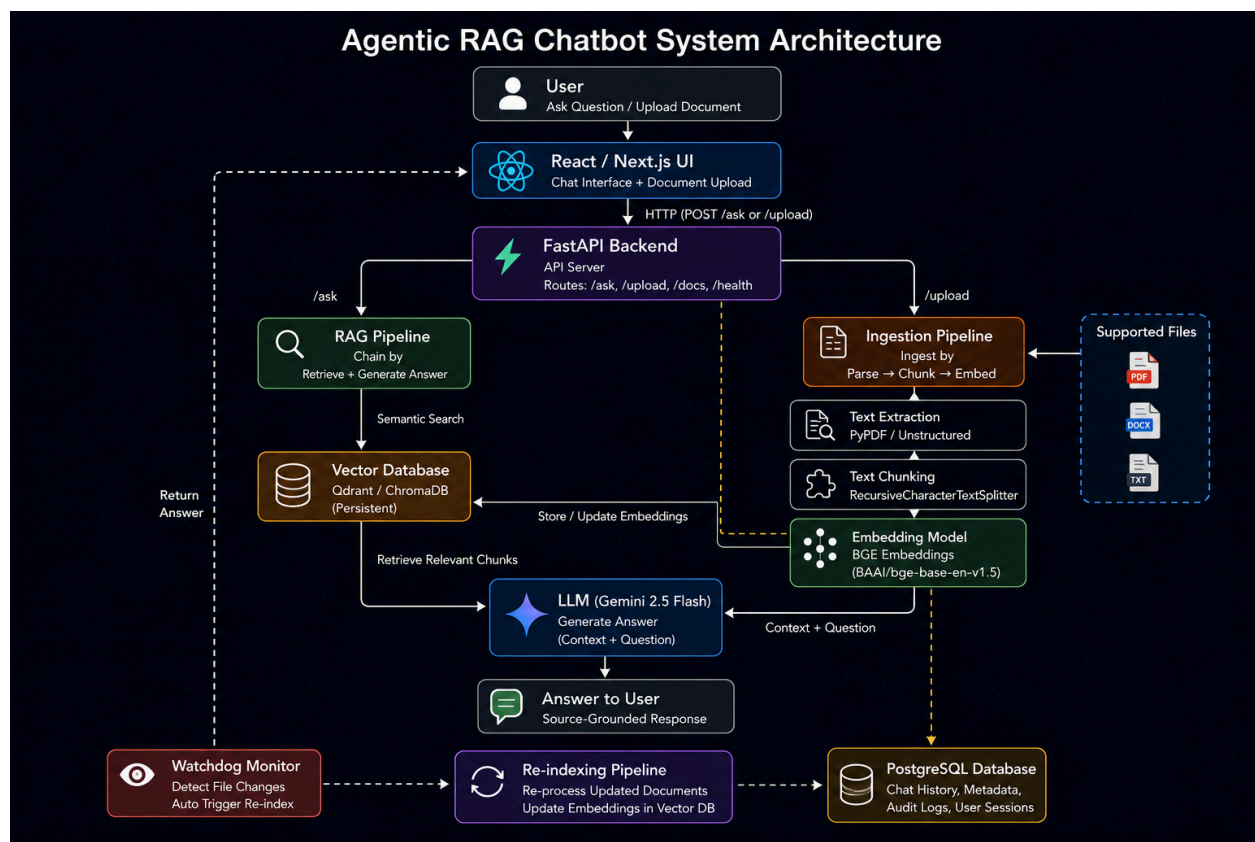
- Multi-format document ingestion (PDF, DOCX, PPTX, XLSX, HTML, Markdown, MSG, TXT)
- Retrieval-Augmented Generation (RAG)
- Agentic Retrieval Loop
- Query Rewriting and Optimization
- Semantic Search
- Context Validation
- Response Verification
- GitHub-based Document Management
- Optional Reindexing/Editing of file

How it works

1. User uploads document
 - User uploads document - (PDFs/PPTs/Excel sheets, etc.) and document is stored in a github repo.
2. Split Documents into Chunks
 - Large documents are broken into chunks
3. Generate Embeddings using BGE
 - Each chunk is converted into a numerical vector (embedding) using an embedding model.
4. Store Embeddings in a Vector Database -
(like ChromaDB, pinecone)
5. User Asks a Question
6. Query Rewriting
7. Generate Query Embedding
 - Convert the user's question into an embedding vector using the same embedding model.
8. Similarity Search will be performed
 - Compare the query embedding against all stored document embeddings.
 - Find the chunks whose vectors are most similar to the query vector.
9. Context Validation

10. Relevant Chunks will be retrieved
12. Send Context to the LLM using Gemini API
13. Generate Final Answer
14. Response Verification

Project Architecture



RAG Pipeline

Documents



Chunking



Embeddings



Vector DB



User Question



Embedding



Similarity Search



Relevant Chunks



LLM



Final Answer

Setup and Installation

Prerequisites

- Python 3.10+
 - Node.js 18+
 - PostgreSQL 14+
 - Git
 - OpenAI/Gemini API Key
 - (Optional) GitHub Personal Access Token for document repository integration
-

Backend Setup

```
python -m venv venv
```

Windows

```
venv\Scripts\activate
```

Install Dependencies

```
pip install -r requirements.txt
```

PostgreSQL Setup

Create a PostgreSQL database:

```
CREATE DATABASE document_agent;
```

Configure Environment Variables

Create a `.env` file in the backend directory:

```
# LLM Configuration
GOOGLE_API_KEY=your_gemini_api_key
```

```
# Database
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=document_agent
POSTGRES_USER=postgres
POSTGRES_PASSWORD=your_password
```

```
# Vector Database
CHROMA_PERSIST_DIR=./vector_store
```

```
# Application
UPLOAD_DIR=./uploads
LOG_LEVEL=INFO
```

Run Backend

```
cd backend
uvicorn main:app --reload --port 8000
```

Backend will be available at:

<http://localhost:8000>

API Documentation:

<http://localhost:8000/docs>

Frontend Setup

```
cd frontend
npm install
```

Start Frontend

```
npm run dev
```

Frontend will be available at:
<http://localhost:3000>

Enable Automatic Re-indexing

Start the Watchdog service:

```
python scripts/watch_documents.py
```

Features:

- Detects new documents
- Detects document updates
- Automatically re-generates embeddings
- Updates vector database and metadata

Verify Installation

Open:

<http://localhost:3000> and test by asking a question